

Programming II

Classes Relationships

Object-Oriented Relationships

- Object oriented programs usually consisted of a **number** of classes
- Only **one** of theses classes will have the main method
- Classes have different relationships
- **association** defines a **relationship** between classes of objects which allows one object instance to cause another to perform an action on its behalf

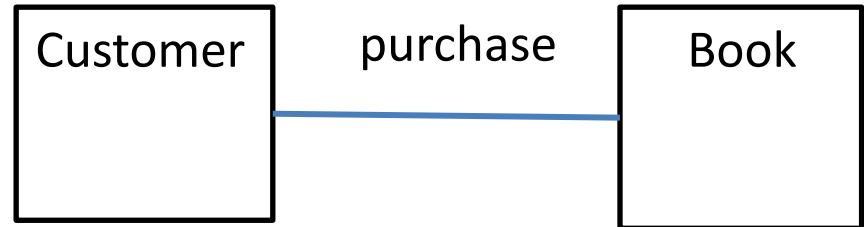
Class Relationships

- The most common relationships:
 - Inheritance: A is-a B 
 - Association: A *uses* B 
 - Aggregation: A *has-a* B
 - Composition: A *part-of* B

Association

- **Association** defines a **relationship** between classes of objects which allows one object instance to cause another to perform an action on its behalf
- One way of implementation is by having object as a **method argument**

A customer purchases a book



```
public class Book
{
    private double price;
    public double
    getPrice()
    {
        return price;
    }
}
```

```
public class Customer
{
    ....
    public void purchase(Book book)
    {
        book.getPrice();
        /////
    }
    ....
}
```

Example Association by passing objects as arguments

```
public class Book{
    private double price;
    private String title;
    public Book(double price,
String title)
{
    this.price=price;
    this.title=title;
}
    public double getPrice()
{
    return price;
}
    public String getTitle()
{
    return title;
}
}
```

300.0

```
public class Customer{
    private double purchaseAmount;
    public void purchase(Book book)
    {
        purchaseAmount +=book.getPrice();
    }
    public double getPurchaseAmount()
    {
        return purchaseAmount ;
    }
}
```

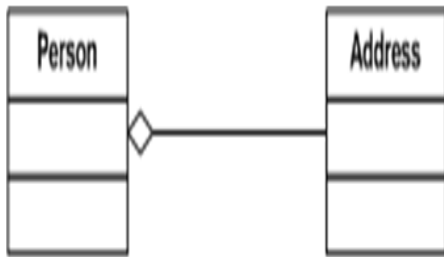
```
public class Main{
    public static void main(String [] args){
        Customer cutomer1=new Customer ();
        Book book1=new Book(100, "Java");
        Book book2=new Book(200, "C#");
        cutomer1. purchase(book1);
        cutomer1. purchase(book2);
        System.out.println(cutomer1.getPurchaseAmount())
;
    }
}
```

Aggregation

- *Aggregation* is a type of *association*
- A connection between two objects where one has a field that refers to the other.
- *Aggregation* is referred to as **weak has a**.
- ***has-a*** relationship is implemented by the having object as member variables

Aggregation Examples

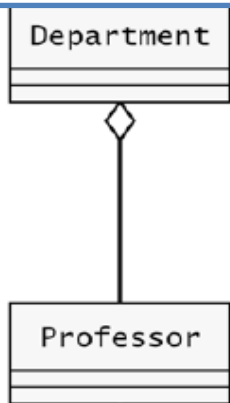
“A person **has a** lawyer “



```
class Person
{
}
}
```

```
class Lawyer
{
    public void
    sue()
}
}
```

“A department **has a** professor“



```
public class Person{
    private Lawyer lawyer;
    Person (Lawyer lawyer){
        this.lawyer=lawyer;
    }
    .....
    lawyer.sue()
}
}
```

```
class Test
{
    ....
    Lawyer mylawyer=new Lawyer ();
    Person p1=new
    Person(myLawyer);
    Person p2=new
    Person(myLawyer);
}
}
```

Example:

```
class Address {  
    int strNum;  
    String city;  
    String state;  
    String country;  
  
    Address(int street, String c, String st, String country) {  
        this.strNum = street;  
        this.city = c;  
        this.state = st;  
        this.country = country;  
    }  
}
```


Example

```
class Student {  
    String stName;  
    Address stAddr;  
  
    Student(String name, Address address){  
        this.stName = name;  
        this.stAddr = address;  
    }  
}
```

Example

```
public class Tester {  
  
    public static void main(String[] args) {  
        Address ad= new Address(10, "Bareilly", "UP", "India");  
        Student st= new Student( "Aashi", ad);  
        System.out.println("Name: "+ st.stName);  
        System.out.println("Street: "+ st.stAddr.strNum);  
        System.out.println("City: "+ st.stAddr.city);  
        System.out.println("State: "+ st.stAddr.state);  
        System.out.println("Country: "+ st.stAddr.country);  
    }  
}
```

Example

Output:

```
Name: Aashi  
Street: 10  
City: Bareilly  
State: UP  
Country: India
```

Composition

- A **composition** is a strong type of aggregation.
- Each component in a composite can belong to just one whole.
- Composition defines the relation “owns a” / “part-of relationship”/ strong has a

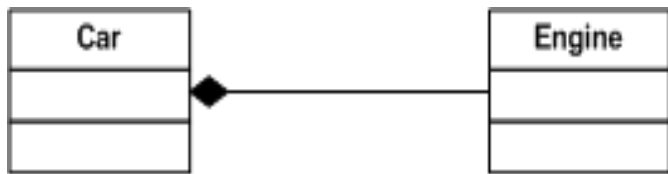


Figure 1 - Composition

```
public class Engine
{....
}
```

```
public class Car{
    Engine e;
    public Car(){
        e=new Engine();
    }
}
```

```
public class Test
{
    public static void main (String []
    arg)
    {
        Car car1 =new Car();
    }
}
```

Example

```
public class Engine
{
    String type;
    int power;
    public Engine (String type, int power){
        this.type = type;
        this.power = power;
    }
}
```

Example

```
public class Car
{ String color;
  Engine engine;
  public Car ( String color, String type, int power){
this.engine = new Engine (type,power);
this.color = color;
}
}
```

Example

```
public class Tester {  
  
    public static void main(String[] args) {  
  
        Car c = new Car("red", "Germany", 100 );  
        System.out.println("Color: "+ c.color);  
        System.out.println("Type: "+ c.engine.type);  
        System.out.println("Power: "+c.engine.power);}  
    }
```

Example

Output:

```
Color: red  
Type: Germany  
Power: 100
```